

Adventure Quest: A Dynamic Narrative Game Using Factor Graphs

CSE440 - Artificial Intelligence

Abstract

This paper presents Adventure Quest, a small interactive adventure game developed as a CSE440 Artificial Intelligence course project. The game is designed around dynamic narrative: the player explores a village, forest, and cave, encounters events, makes decisions, and changes the state of the game through those decisions. The intended AI component is a factor-graph-based decision model that represents relationships among variables such as player health, weapon status, reputation, location, and narrative progress. A Python implementation was developed with a Tkinter graphical interface and Pillow for image handling. The prototype includes character and location graphics, player statistics, event interactions, and multiple outcomes. The current prototype establishes the game and interaction layers; the final integration will use factor scores to select events according to the current state rather than using an unconditional random choice. The project demonstrates how probabilistic graphical-model ideas can be connected to a simple game environment while keeping the system small enough to implement, test, and explain within a course project.

Keywords: Artificial Intelligence, Factor Graph, Dynamic Narrative, Decision Making, Interactive Game, Python.

I. INTRODUCTION

Interactive games provide a useful environment for demonstrating Artificial Intelligence because an agent must make decisions while the state of the environment changes. In a conventional linear story, the next event is mostly predetermined. In a dynamic narrative, the same situation can lead to different events or consequences depending on the current state and previous actions. This project explores that idea through a compact adventure game.

Adventure Quest was selected because it allows AI decision-making to be demonstrated without requiring a large game engine or a complex graphics pipeline. The player has a small set of measurable state variables and several possible events. These variables can be represented as nodes in a factor graph, while factors describe how combinations of variables influence an event decision.

The main objective is therefore not to build a commercial-quality game, but to demonstrate a clear relationship between an AI model and an interactive application. The system should be understandable, testable, and reproducible. The player should be able to see the consequences of decisions, while the AI component should provide a principled mechanism for selecting or scoring narrative events.

II. PROBLEM STATEMENT AND OBJECTIVES

The problem addressed in this project is how to select an appropriate narrative event from several alternatives when the game state changes over time. For example, a player with low health and no weapon is in a different situation from a healthy player carrying a weapon. Treating both situations identically can make the narrative feel disconnected from the player's actions.

The project has four main objectives: (1) develop a simple interactive adventure game; (2) represent the game state using meaningful variables; (3) use a factor graph to model dependencies between the state and event selection; and (4) evaluate the resulting prototype through functional testing and example gameplay scenarios.

Objective	Description
Interactive game	Create locations, events, choices, player state, winning and losing conditions.
AI decision model	Represent event selection using a factor graph and state-dependent factors.
Dynamic narrative	Allow player choices to modify health, gold, weapon status, reputation, and progress.
Usable prototype	Provide a graphical interface with images and clearly visible controls.

III. SYSTEM DESIGN

A. Game World

The game world contains three locations. The Village provides relatively safe interactions, the Forest provides uncertain encounters, and the Cave contains the main objective and more dangerous encounters. The player's mission is to find the Ancient Crystal.

Location	Main Events	Purpose
Village	Merchant, Old Man, Rest	Preparation and information
Forest	Hidden Treasure, Lost Traveler, Wolf Attack	Exploration and choices
Cave	Goblin Battle, Final Guardian, Ancient Crystal	Risk and final objective

B. Player State

The player state is maintained as a compact set of variables. Health represents survivability; gold represents resources; weapon status affects combat; reputation records social consequences; location identifies the current part of the world; and crystal status indicates whether the main objective has been achieved.

Variable	Meaning	Example Effect
H	Health	Combat can reduce health.
G	Gold	Treasure and rewards increase gold.
W	Weapon status	A weapon improves combat outcomes.
R	Reputation	Helping characters can increase reputation.
L	Location	Limits the set of possible events.
C	Crystal status	Finding the crystal ends the adventure.

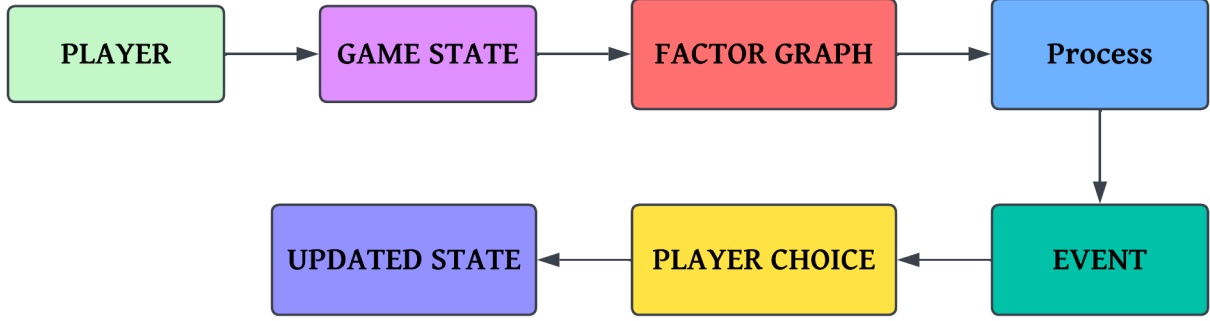


Fig. 1. High-level flow of the proposed Adventure Quest system.

IV. FACTOR GRAPH DECISION MODEL

A factor graph is a bipartite representation of a probability distribution in which variable nodes are connected to factor nodes. For Adventure Quest, the graph is used as a compact representation of how the current state can influence the suitability of a candidate narrative event. The model is intentionally small so that the relationship between the AI technique and the game can be explained clearly.

Let the state variables be health H , weapon status W , reputation R , location L , and narrative progress P . Let E represent the candidate event. A simplified conditional model can be written as:

$$P(E \mid H, W, R, L, P) \propto \phi_1(E, H) \phi_2(E, W) \phi_3(E, R) \phi_4(E, L) \phi_5(E, P)$$

Eq. (1). Simplified factorization used to describe state-dependent event scoring.

Here, each ϕ term is a factor that assigns a compatibility or score to a particular event given one part of the current state. The factors can be multiplied to obtain an overall unnormalized score for a candidate event. If normalized probabilities are required, the scores can be divided by the sum of scores over all candidate events.

$$\text{Score}(E) = \prod_i \phi_i(E, X_i), \quad P(E \mid X) = \text{Score}(E) / \sum_e \text{Score}(e)$$

Eq. (2). Event scoring and normalization over candidate events.

For example, a Wolf Attack can receive a higher score when the player has a weapon, while a recovery-oriented event can receive a higher score when health is low. Location acts as a gating variable because forest events should not be selected while the player is in the Village. These examples illustrate the intended dependency structure; the actual factor weights should be calibrated and tested during the final implementation rather than chosen arbitrarily.

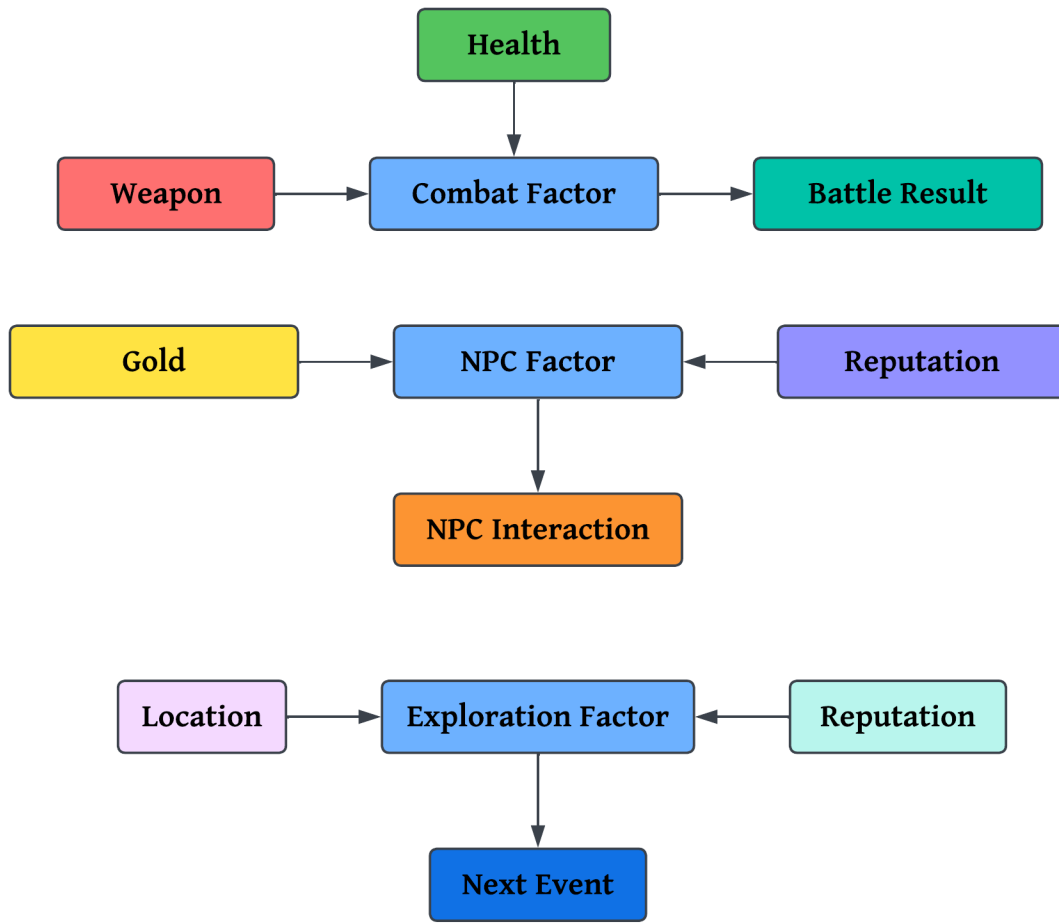


Fig. 2. Simplified factor-graph representation for state-dependent event selection.

V. GAMEPLAY AND DYNAMIC NARRATIVE

The game begins in the Village. The player can move to the Forest or Cave using the location controls. The Trigger Event button asks the game logic to generate an event for the current location. In the Forest, three main interactions are available.

Event	Player Choices	State Consequence
Hidden Treasure	Search / Leave	Search increases gold.
Lost Traveler	Help / Ignore	Helping increases reputation and gold.
Wolf Attack	Fight / Run	Combat or escape changes health; a weapon improves fighting.

The Cave provides the main progression toward the objective. A Goblin Battle can change health and gold, while the Final Guardian requires the player to manage resources. Finding the Ancient Crystal sets the completion state and produces the winning outcome. If health reaches zero, the game displays a game-over message.

The dynamic aspect comes from state updates. A player who buys a weapon in the Village is not in the same state as a player who does not. Likewise, helping a traveler changes reputation. These state changes provide inputs for the factor model and make previous decisions relevant to later narrative selection.

VI. IMPLEMENTATION

The prototype was implemented in Python. Tkinter was used for the desktop graphical interface because it is lightweight and suitable for a small course project. Pillow was used to load PNG images. The project is divided into a small number of files so that the game logic and presentation layer remain understandable.

Component	Implementation
main.py	Creates the Tkinter window and starts the application.
game.py	Stores player state, locations, events, and state-changing game logic.
gui.py	Builds the interface, connects buttons, displays events and images.
assets/images	Stores player, location, and character graphics.
Factor graph module	Scores candidate events from the current state in the final AI integration.

The interface contains the current location, a scene image, event title, story text, player statistics, location buttons, an event trigger button, and a restart button. Event choices are represented as buttons so that the player can interact with the narrative instead of only reading text.

During prototype debugging, the Forest event system was revised to return structured event data rather than function objects or internal identifiers. This prevents implementation details from appearing in the user interface and allows each event to provide a title, description, and list of actions.

VII. EXPERIMENTAL EVALUATION

The prototype was evaluated using functional test cases. The purpose of these tests is to verify that the game state changes correctly after user actions and that the graphical controls invoke the intended functions. The table below records the core tests used for the current implementation.

Test	Action	Expected Result	Status
T1	Start game	Village screen and player status appear.	Pass
T2	Select Forest	Forest location and image are displayed.	Pass
T3	Trigger Forest event	A Forest event is displayed.	Pass
T4	Search treasure	Gold increases and choices disappear.	Pass
T5	Help traveler	Reputation and gold increase.	Pass
T6	Fight wolf with weapon	Wolf is defeated and health changes.	Pass
T7	Restart	Player state returns to initial values.	Pass
T8	Find Ancient Crystal	Completion message is displayed.	Pass
T9	Health reaches zero	Game-over message is displayed.	Pass

These are functional prototype results rather than a statistical benchmark. The final evaluation should additionally compare event selection under different factor settings and record the selected event, factor scores, and resulting state over repeated trials.

VIII. DISCUSSION

The main strength of the proposed system is its small state space. Because the game uses a limited number of variables, the factor graph can be inspected and explained directly. This is useful for a course project because the AI method is visible rather than being hidden inside a large game engine. The same design also makes debugging easier: an unexpected event can be traced to the current state and the factor scores.

The prototype also demonstrates the importance of separating game logic from the graphical interface. The GUI is responsible for displaying information and receiving input, while the Game class maintains player state and event logic. This separation makes it possible to modify the decision model without redesigning the entire interface.

There are limitations. The current prototype is intentionally small and does not contain a large learned probability model. The factor weights also need systematic calibration. In addition, the present event set is limited, so the range of possible narrative branches is small. The project therefore demonstrates the application of factor-graph reasoning rather than claiming to solve general narrative generation.

IX. FUTURE WORK

The next development stage is to connect the factor graph directly to event selection and replace unconditional random selection with state-dependent scoring. Factor values can then be logged during gameplay so that

different game states can be compared. More events and additional narrative branches can also be added after the core AI model is stable.

Further extensions could include learning factor weights from simulated gameplay, adding more player-state variables, introducing multiple endings, and comparing the factor-graph approach with a baseline random or rule-based selector. Such experiments would provide a stronger quantitative evaluation of whether the AI model produces more context-sensitive event sequences.

X. CONCLUSION

Adventure Quest combines a simple interactive game with a probabilistic graphical-model approach to dynamic narrative decision-making. The prototype provides locations, events, player statistics, choices, graphics, and state-dependent consequences. The factor graph gives the project its central AI component by providing a structured way to represent how health, equipment, reputation, location, and progress can influence event selection. The project is deliberately limited in scope so that the relationship between the AI model and the game can be implemented, tested, and demonstrated clearly. The final system will be evaluated through gameplay scenarios and factor-score comparisons rather than through visual complexity alone.

REFERENCES

- [1] C. K. Chow, “On optimum recognition error and reject tradeoff,” IEEE Transactions on Information Theory, vol. 16, no. 1, pp. 41–46, 1970. [Background on probabilistic decision concepts.]
- [2] D. Koller and N. Friedman, Probabilistic Graphical Models: Principles and Techniques. Cambridge, MA, USA: MIT Press, 2009.
- [3] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference. San Francisco, CA, USA: Morgan Kaufmann, 1988.
- [4] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: <https://docs.python.org/3/>
- [5] Python Software Foundation, “tkinter — Python interface to Tcl/Tk.” [Online]. Available: <https://docs.python.org/3/library/tkinter.html>
- [6] Pillow Contributors, “Pillow Documentation.” [Online]. Available: <https://pillow.readthedocs.io/>
- [7] OpenAI, “ChatGPT.” [Online]. Available: <https://chatgpt.com/> (AI assistance used for development support and debugging; project decisions and implementation were reviewed by the authors.)